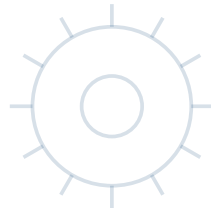


Linux for Computational Chemistry

Lecture 3: Shell Scripting with Bash

Emilio Martínez Núñez

Master in Theoretical and Computational Chemistry



Outline

Why script?

Variables and arguments

Conditionals

Loops

Functions

Capstone script

Debugging

Why script?

From one-liners to scripts

- Last class you built pipelines like `grep | awk | sort` on the command line
- A **script** saves that logic to a file: repeatable, shareable, versionable
- Typical use in computational chemistry: process 50, 500, or 5000 conformers with the same steps, with zero copy-paste

Anatomy of a script

```
#!/bin/bash  
# check_energy.sh -- print the SCF energy of a Gaussian log file  
grep "SCF Done" opt.log
```

- Line 1: the **shebang** — tells the OS which interpreter to use
- Lines starting with # are comments
- Run it with `bash check_energy.sh`, or make it executable and run `./check_energy.sh`

Making a script executable

```
chmod +x check_energy.sh  
./check_energy.sh
```

Note

The leading `./` is required — unlike Windows, the current directory is (deliberately) not searched automatically for commands.

Variables and arguments

Variables

```
name="butanol" # assignment: NO spaces around =  
echo "Molecule: $name" # reference with $  
echo "Molecule: ${name}_opt" # braces avoid ambiguity  
count=5  
echo $((count + 1)) # arithmetic needs $(( ))
```

Tip

Always double-quote variables you use as file paths: "\$file", not \$file — protects against spaces and empty values.

Command substitution

```
today=$(date +%Y-%m-%d)
n_confs=$(ls conf*.xyz | wc -l)
echo "Found $n_confs conformers on $today"
```

- \$(command) runs command and substitutes its output as text
- This is how scripts capture the result of any command into a variable

Script arguments

```
#!/bin/bash
# usage: ./check_energy.sh mylog.log
echo "Script name : $0"
echo "First arg : $1"
echo "Num args : $# "
echo "All args : $@"
```

Exercise 3.1 — A parametrized script

Exercise 3.1

Write a script `show_energy.sh` that:

1. Takes one argument: the name of a log file.
2. Prints a message "Checking <filename>...".
3. Prints every line of that file containing "SCF Done".
4. Test it on at least two different log files without editing the script.

Solution

```
#!/bin/bash  
echo "Checking $1..."  
grep "SCF Done" "$1"
```

```
chmod +x show_energy.sh  
./show_energy.sh conf1.log  
./show_energy.sh conf2.log
```

Conditionals

```
if grep -q "Normal termination" "$1"; then
    echo "$1 finished OK"
else
    echo "$1 did NOT finish normally"
fi
```

- `grep -q` runs silently and only sets an exit status (success/failure)
- Common file tests: `-f file` (exists, regular file), `-d dir` (exists, directory), `-e path` (exists, any type)
- Numeric: `-eq -ne -lt -le -gt -ge`; string: `= != -z (empty) -n (not empty)`

Exercise 3.2

Write `status.sh`, taking one log-file argument, that:

1. Prints `$1: DONE` if the file contains "Normal termination".
2. Otherwise prints `$1: FAILED`.
3. Additionally, before checking, verifies with `-f` that the file actually exists, printing `$1: NOT FOUND` and stopping if it does not (hint: `exit 1`).

Solution

```
#!/bin/bash
if [ ! -f "$1" ]; then
    echo "$1: NOT FOUND"
    exit 1
fi
if grep -q "Normal termination" "$1"; then
    echo "$1: DONE"
else
    echo "$1: FAILED"
fi
```


Loops

for loops

```
for f in conf*.log; do
    echo "Processing $f"
    grep "SCF Done" "$f"
done
```

```
for i in 1 2 3 4 5; do
    echo "Conformer number $i"
done
```

while loops

```
while read -r line; do  
    echo "Line: $line"  
done < coords.xyz
```

Note

for f in *.log loops over *files*; while read loops over *lines* inside one file.
Knowing which one you need is half the battle.

Exercise 3.3 — Batch energy extraction

Exercise 3.3

Write `extract_all.sh` (no arguments) that:

1. Loops over every `*.log` file in the current directory.
2. For each, extracts the *last* SCF energy value (hint: pipe `grep` through `tail -1`, then `awk` for the field).
3. Appends a line `filename,energy` to a file `summary.csv` for each conformer.
4. Make sure re-running the script does not just keep appending duplicate rows forever (hint: remove/recreate `summary.csv` at the start).

Solution

```
#!/bin/bash
rm -f summary.csv
for f in *.log; do
    energy=$(grep "SCF Done" "$f" | tail -1 | awk '{print $5}')
    echo "$f,$energy" >> summary.csv
done
echo "Wrote summary.csv"
```

Functions

Functions

```
get_energy() {  
    local file="$1"  
    grep "SCF Done" "$file" | tail -1 | awk '{print $5}'  
}  
  
for f in *.log; do  
    e=$(get_energy "$f")  
    echo "$f -> $e"  
done
```

Note

`local` keeps a variable scoped to the function — good practice to avoid clashes as scripts grow.

Capstone script

Exercise 3.4

Combine everything into one script `check_jobs.sh` (no arguments) that, for every `*.log` file in the current directory:

1. Reports `DONE`, `FAILED`, or `RUNNING` (hint: a job still running may have neither termination string — treat "neither found" as `RUNNING`).
2. For `DONE` jobs only, also extracts the final SCF energy using a function.
3. Prints a final one-line summary: "X done, Y failed, Z running (out of N)".
4. Writes the per-file energies of `DONE` jobs to `summary.csv`.

Solution 3.4 (part 1)

Solution

```
#!/bin/bash
get_energy() {
    grep "SCF Done" "$1" | tail -1 | awk '{print $5}'
}

rm -f summary.csv
done_n=0; failed_n=0; running_n=0; total=0
```

Solution 3.4 (part 2)

Solution

```
for f in *.log; do
    total=$((total + 1))
    if grep -q "Normal termination" "$f"; then
        e=$(get_energy "$f")
        echo "$f: DONE ($e)"
        echo "$f,$e" >> summary.csv
        done_n=$((done_n + 1))
    elif grep -q "Error termination" "$f"; then
        echo "$f: FAILED"
        failed_n=$((failed_n + 1))
    else
        echo "$f: RUNNING"
        running_n=$((running_n + 1))
    fi
done
```

Debugging

Debugging scripts

```
bash -x check_jobs.sh # print every command as it executes
```

```
#!/bin/bash  
set -e # stop the script on the first failing command  
set -u # error out on use of an undefined variable
```

Tip

Adding `set -e` and `set -u` at the top of production scripts catches silent mistakes early — a very good habit once scripts feed into HPC job submission (Lecture 4).

- Variables, arguments, command substitution
- `if/test` for conditionals, `for/while` for loops
- Functions to structure and reuse logic
- You now have a working `check_jobs.sh` — the backbone of the HPC workflow in Lecture 4

Exercise 3.5 (Homework)

Extend `check_jobs.sh` to:

1. Accept an optional argument: a directory to scan (default to `.` if none given).
2. Skip printing per-file lines for `RUNNING` jobs and instead only count them.
3. Sort `summary.csv` by energy before the script finishes (reuse what you learned about `sort` in Lecture 2).